



EMHIP — Ethnicity & Mental Health Improvement Project

EMHIP Deployment and Operations Runbook

Version 1.0

30 September 2026

For: whoever runs the EMHIP production server

Contents

1. Environments
2. How deployment works
3. How to deploy a change
4. Verifying a deploy
5. Rolling back
6. Backups
7. Restoring
8. Logs and troubleshooting
9. Restarting services
10. TLS and certificates
11. Rotating secrets
12. Admin access
13. Setting up email
14. Setting up document storage
15. Importing legacy data
16. Monitoring suggestions
17. Routine checks

This runbook is for the person who looks after the live EMHIP server. It explains how a change reaches production, how to check a deployment, how to roll back, how backups work and how to restore one, and how to deal with the most likely problems. Every command was taken from, or checked against, the scripts in the repository (`deploy/`, `docker-compose.yml`, `docker-compose.prod.yml`) on 30 September 2026. How the code itself works is in document 05, the Technical Handover.

1. Environments

Item	Value
Production URL	https://emhip.brainshub.co.uk
Host	81.0.221.206 , Ubuntu 24.04
Access	ssh root@81.0.221.206
Application directory	/opt/emhip (a git checkout of <code>main</code> , used only as a deploy target)
Compose files	docker-compose.yml with docker-compose.prod.yml layered on top
Secrets	/opt/emhip/.env (mode 600, never in git)
Database	SQL Server 2022, Express edition by default, in the Docker volume <code>sqlserver-data</code> (Docker names it <code>emhip_sqlserver-data</code>)
Backups	/opt/emhip-backups
Uploaded documents	/opt/emhip-documents while the storage setting is Local
Host nginx site	/etc/nginx/sites-available/emhip (not in the repository)
In production since	17 August 2026

There is no staging environment. Local development environments are described in document 05, section 11.

1.1 Network layout

Port	Bound to	Service
22	Public, rate-limited by UFW	SSH
80, 443	Public	Host nginx: TLS (Let's Encrypt) and proxy to the client container
8080	127.0.0.1	client container (nginx serving the app, proxying <code>/api/</code> and <code>/hubs/</code> to the API)
5299	127.0.0.1	api container (port 8080 inside)
1433	127.0.0.1	sqlserver container

Ports published by Docker bypass UFW, so binding them to `127.0.0.1` in `docker-compose.prod.yml` is what keeps SQL Server and the API off the internet. Do not change those bindings. To reach SQL Server from a workstation, use an SSH tunnel: `ssh -L 14330:127.0.0.1:1433 root@81.0.221.206` , then connect a SQL client to `localhost,14330` as `sa` with the password from `.env` .

The host was hardened once, by hand, when it was built: Docker log rotation and `live-restore` , UFW (22 rate-limited, 80, 443) and fail2ban, 4 GB swap with `vm.swappiness=10` , BBR congestion control, unattended

security upgrades, and SQL Server capped at 4 GB RAM through `MSSQL_MEMORY_LIMIT_MB`.

1.2 Shell set-up used in this runbook

Most commands below assume you are in `/opt/emhip` with this variable set:

```
cd /opt/emhip
COMPOSE="docker compose -f docker-compose.yml -f docker-compose.prod.yml"
```

Commands that talk to SQL Server also need the `sa` password in the shell. This loads every secret from `.env` into the current shell, so close the shell when you have finished:

```
set -a; source /opt/emhip/.env; set +a
```

2. How deployment works

Deployment is pull-based and automatic. Pushing a commit to `main` on GitHub is the deployment.

1. `emhip-deploy.timer` starts `emhip-deploy.service` 2 minutes after boot and then 60 seconds after each run finishes (plus up to 10 seconds of random delay).
2. The service runs `/opt/emhip/deploy/deploy.sh`. It takes a lock (`/var/lock/emhip-deploy.lock`), so only one deploy runs at a time; a run that finds the lock taken logs "another deploy is running; skipping" and exits.
3. The script runs `git fetch origin main` and compares `HEAD` with `origin/main`. If they match, it exits (unless run with `--force`).
4. It takes a full database backup with `deploy/backup.sh pre-deploy`. If the backup fails while the `sqlserver` container is running, the deploy stops with "ABORT: sqlserver is running but backup failed". If no database is running (first deploy), it continues with a warning.
5. It runs `git reset --hard origin/main`. Local edits to tracked files on the server are thrown away; `.env` is ignored by git and is not touched.
6. It copies `deploy/systemd/*.service` and `*.timer` to `/etc/systemd/system/` and runs `systemctl daemon-reload`. It does not enable new units; that is a manual step.
7. It runs `docker compose build` for the `api`, `workers` and `client` images while the old containers keep serving. (The `seeder` image is behind a Compose profile and is not built.)
8. It runs `docker compose up -d --remove-orphans`, which recreates the containers whose images or settings changed. The new API container applies any pending EF Core migrations and runs the seeders (built-in roles, option lists, email templates) before it starts serving.
9. It runs `docker image prune -f` and logs a final line such as `[deploy] ... done: api=running client=running sqlserver=running workers=running`.

A deploy starts within about a minute of the push. The build then takes several minutes, and the app is unavailable only while containers restart and migrations run. The service is allowed 45 minutes (`TimeoutStartSec=45min`) before `systemd` stops it.

Warning: nothing checks a commit before it goes live. No tests run on push. Whoever can push to `main` can change production.

The server fetches from `https://github.com/azuree123/emhip_new.git` (check with `git -C /opt/emhip remote -v`). If the repository is private, the server needs credentials that can read it; if those are revoked,

deploys silently stop happening, so check the deploy log after changing GitHub access.

3. How to deploy a change

1. Make the change on a branch and run the checks in document 05, section 11.5: `dotnet test Emhip.slnx`, `npx ng build` and `npx ng test --watch=false` in `client`.
2. If the change includes a database migration, read it for data loss and make sure it is backwards compatible (document 05, section 12).
3. Pick a quiet time if the change includes a migration or touches sign-in, uploads or urgent cases.
4. Merge into `main` and push: `git push origin main`.
5. Watch it on the server with `journalctl -u emhip-deploy.service -f`, or wait about 10 minutes.
6. Verify it (section 4).

To redeploy the current commit without a new push (for example after changing `.env` or when a build failed for a temporary reason):

```
ssh root@81.0.221.206 '/opt/emhip/deploy/deploy.sh --force'
```

To pause automatic deployment, run `systemctl stop emhip-deploy.timer`; start it again with `systemctl start emhip-deploy.timer`. A stopped timer starts again after a reboot because it is enabled; use `systemctl disable --now emhip-deploy.timer` to pause across reboots, and `systemctl enable --now emhip-deploy.timer` to undo that.

4. Verifying a deploy

These three checks need no server access:

```
# 1. The API is up: expect {"status":"ok"} and HTTP 200
curl -s -w ' HTTP %{http_code}\n' https://emhip.brainshub.co.uk/api/health

# 2. A route added in this release is live: expect 401 once deployed (404 before)
curl -s -o /dev/null -w '%{http_code}\n' https://emhip.brainshub.co.uk/api/<new-authorized-route>

# 3. The app bundle changed: the main-XXXXXXX.js name differs from before the deploy
curl -s https://emhip.brainshub.co.uk/ | grep -o 'main-[A-Za-z0-9]*\.js'
```

`/api/health` does not touch the database, so a 200 only proves the API process is running. Check 2 works for any route protected by `[Authorize]`: an unauthenticated request gets 401 from a route that exists and 404 from one that does not.

On the server, also check:

- `git -C /opt/emhip log -1 --oneline` shows the commit you pushed.
- `journalctl -u emhip-deploy.service -n 20` ends with the `done:` line and all four services `running`.
- `$COMPOSE ps` shows `api`, `client`, `sqlserver` and `workers` up, and `api` not restarting.
- `$COMPOSE logs --since 15m api` has no migration errors or repeated exceptions.

Then sign in and open the guest list, a guest record, Urgent Cases and Reports. Dashboard counts are recalculated every 5 minutes, so check them after that.

5. Rolling back

`deploy.sh` has no option to deploy an older commit: it always resets to `origin/main`. There are two ways back.

5.1 Preferred: revert on GitHub

1. On a workstation, revert the bad commit: `git revert <commit>` (for several commits, `git revert --no-commit <first>^.. then git commit).`
2. Push to `main`. The normal pipeline deploys the reverted code, taking a backup first.
3. Verify (section 4).

This keeps history clean and the server stays a pure deploy target.

Warning: reverting code does not undo a database migration that has already run. Older code usually works against a schema with extra columns, but not always. If the bad release included a migration, prefer a forward fix, or restore the pre-deploy backup (section 7), accepting that data entered since the deploy is lost.

5.2 Emergency: pin the server to an earlier commit

Use this only when you cannot push to GitHub quickly.

1. Stop automatic deploys, or the timer will reset the checkout to `origin/main` within a minute: `systemctl stop emhip-deploy.timer`.
2. Take a backup, because this path skips the automatic one: `/opt/emhip/deploy/backup.sh pre-rollback`.
3. Check out the last good commit: `cd /opt/emhip && git reset --hard <good-commit>`.
4. Rebuild and restart: `$COMPOSE build && $COMPOSE up -d --remove-orphans`.
5. Verify (section 4).
6. Fix `main` on GitHub (revert or forward fix), then start the timer again: `systemctl start emhip-deploy.timer`. The next run deploys `main`.

Warning: while the deploy timer is stopped, nothing pushed to `main` reaches production. Write down that you stopped it, and start it again.

5.3 Rolling back the database

Each deploy leaves a backup named `emhip-pre-deploy-YYYYMMDD-HHMMSS.bak.gz` in `/opt/emhip-backups`, taken just before the code changed. Restoring it (section 7.1) returns the database to that moment; everything entered since is lost.

6. Backups

6.1 What `deploy/backup.sh` does

1. Loads `.env` for the `sa` password.
2. Makes sure `/opt/emhip-backups` exists and is writable by the SQL Server user inside the container (UID 10001).

3. Runs `BACKUP DATABASE [Emhip] ... WITH INIT, CHECKSUM` inside the `sqlserver` container, writing to `/var/opt/mssql/backup`, which is the host's `/opt/emhip-backups`.
4. Compresses the file with `gzip` on the host (Express edition cannot compress backups itself).
5. If `/opt/emhip-documents` is not empty, writes a `tar.gz` of it next to the database backup.
6. Deletes database backups and document archives older than 14 days.

Item	Detail
Command	<code>/opt/emhip/deploy/backup.sh [tag]</code> ; the tag defaults to <code>scheduled</code>
Database file	<code>/opt/emhip-backups/emhip-TAG-YYYYMMDD-HHMMSS.bak.gz</code>
Documents file	<code>/opt/emhip-backups/emhip-documents-TAG-YYYYMMDD-HHMMSS.tar.gz</code>
Nightly	03:00 server time by <code>emhip-backup.timer</code> (up to 5 minutes random delay; runs at next boot if the server was off)
Before each deploy	Run by <code>deploy.sh</code> with the tag <code>pre-deploy</code>
Retention	14 days, by file age, applied at the end of every run
Logs	<code>journalctl -u emhip-backup.service</code> (nightly); pre-deploy backups log to <code>emhip-deploy.service</code>

For a manual backup, for example before a data import or a risky change, run `/opt/emhip/deploy/backup.sh manual`.

6.2 What is not backed up

- `/opt/emhip/.env`. Keep a copy of every value in the organisation's password manager.
- The host nginx site configuration and the Let's Encrypt certificates.
- Documents stored with a cloud provider (S3, Azure, Google). They rely on the provider's own durability; turn on versioning or backups there.
- Anything off the server. The backups sit on the same disk as the database, unencrypted. Copy them off-site (and encrypt them) as a priority; see also the UK GDPR register, document 09.

7. Restoring

Warning: there is no restore script in the repository. The procedures below are derived from the restore command in the header of `deploy/backup.sh` and standard SQL Server commands. They have not been rehearsed as part of writing this document; rehearse section 7.3 before you need 7.1.

7.1 Restore the production database

Step 1: Choose the backup

```
ls -lt /opt/emhip-backups | head -20
```

Note the database file name without `.gz`, for example `emhip-scheduled-20260930-030012.bak`, and the documents archive with the same date and time if there is one.

Step 2: Stop deployments and the application

```
systemctl stop emhip-deploy.timer
cd /opt/emhip
COMPOSE="docker compose -f docker-compose.yml -f docker-compose.prod.yml"
$COMPOSE stop api workers
```

Users get errors from this point until step 7. Tell them first.

Step 3: Back up the current state

```
/opt/emhip/deploy/backup.sh pre-restore
```

This lets you undo the restore if you picked the wrong file.

Step 4: Unpack the chosen backup

```
BACKUP=emhip-scheduled-20260930-030012.bak
gunzip -k /opt/emhip-backups/$BACKUP.gz
chown 10001:0 /opt/emhip-backups/$BACKUP
```

`-k` keeps the compressed copy. The `chown` makes sure SQL Server (UID 10001 in the container) can read the file.

Step 5: Verify the file, then restore

```
set -a; source /opt/emhip/.env; set +a
$COMPOSE exec -T sqlserver /opt/mssql-tools18/bin/sqlcmd -C -S localhost -U sa -P
"$MSSQL_SA_PASSWORD" -b \
-Q "RESTORE VERIFYONLY FROM DISK='/var/opt/mssql/backup/$BACKUP' WITH CHECKSUM"
$COMPOSE exec -T sqlserver /opt/mssql-tools18/bin/sqlcmd -C -S localhost -U sa -P
"$MSSQL_SA_PASSWORD" -b \
-Q "ALTER DATABASE [Emhip] SET SINGLE_USER WITH ROLLBACK IMMEDIATE; RESTORE DATABASE
[Emhip] FROM DISK='/var/opt/mssql/backup/$BACKUP' WITH REPLACE, CHECKSUM; ALTER DATABASE
[Emhip] SET MULTI_USER;"
```

If the restore fails part-way, the database may be left in single-user mode. Fix the cause, then run the same `sqlcmd` command with `-Q "ALTER DATABASE [Emhip] SET MULTI_USER"`.

Step 6: Restore documents (Local storage only)

```
tar -xzf /opt/emhip-backups/emhip-documents-scheduled-20260930-030012.tar.gz -C /opt/emhip-
documents
```

This writes the archived files back over the directory. Files uploaded after the backup stay on disk but are no longer referenced by the database, which is harmless.

Step 7: Start the application and deployments

```
$COMPOSE up -d
systemctl start emhip-deploy.timer
```

If the running code is newer than the backup, the API applies the missing migrations as it starts.

Step 8: Check the result

- Run the checks in section 4 and sign in.
- Confirm the data is from the expected time, for example with `SELECT MAX(OccurredAt) FROM Contacts` and `SELECT MAX(RegisteredAt) FROM Guests`.
- Wait 5 minutes for dashboard counts to be recalculated.
- Delete the unpacked file and keep the `.gz`: `rm /opt/emhip-backups/$BACKUP`.
- Record what you restored, when and why.

7.2 Rebuilding on a new server

Use this if the host is lost. You need the backup files, the `.env` values and DNS control for `emhip.brainshub.co.uk`.

1. Install Ubuntu 24.04, Docker Engine with a recent Compose plugin (the production file uses `!override` tags, which need Compose 2.24 or later), nginx and certbot. Apply the host hardening listed in section 1.
2. Clone the repository to `/opt/emhip` and create `/opt/emhip/.env` (mode 600) with every variable from `.env.example`, including `PUBLIC_ORIGIN=https://emhip.brainshub.co.uk`. The `sa` password can be new; the restored database does not depend on it.
3. Copy the backup files to `/opt/emhip-backups` and create `/opt/emhip-documents`; unpack the documents archive into it.
4. Start only SQL Server: `$COMPOSE up -d sqlserver`, and wait until `$COMPOSE ps` shows it healthy.
5. Restore the database with the step 5 command in section 7.1, leaving out the two `ALTER DATABASE` statements (the database does not exist yet). Restore before the API first starts, or it will create an empty database with a new first admin.
6. Build and start everything: `$COMPOSE up -d --build`.
7. Configure the host nginx site to proxy `emhip.brainshub.co.uk` to `http://127.0.0.1:8080`, pass WebSocket upgrade headers for `/hubs/`, and set `client_max_body_size` large enough for uploads. Then run `certbot --nginx -d emhip.brainshub.co.uk`.
8. Install the timers: `cp deploy/systemd/*.service deploy/systemd/*.timer /etc/systemd/system/ && systemctl daemon-reload && systemctl enable --now emhip-deploy.timer emhip-backup.timer`.
9. Point DNS at the new host and run the checks in section 4.

7.3 Test restore to a scratch database

Do this monthly. It proves a backup is readable without touching the live database. It needs free disk space for a second copy of the database.

Step 1: Unpack a recent backup

Follow step 1 and step 4 of section 7.1, and load `.env` into the shell.

Step 2: Find the logical file names

```
$COMPOSE exec -T sqlserver /opt/mssql-tools18/bin/sqlcmd -C -S localhost -U sa -P
"$MSSQL_SA_PASSWORD" \
-Q "RESTORE FILELISTONLY FROM DISK='/var/opt/mssql/backup/$BACKUP'"
```

Note the two values in the `LogicalName` column (data and log).

Step 3: Restore under another name and check it

```
$COMPOSE exec -T sqlserver /opt/mssql-tools18/bin/sqlcmd -C -S localhost -U sa -P
"$MSSQL_SA_PASSWORD" -b \
-Q "RESTORE DATABASE [Emhip_RestoreTest] FROM DISK='/var/opt/mssql/backup/$BACKUP' WITH
MOVE 'DATA_LOGICAL_NAME' TO '/var/opt/mssql/data/Emhip_RestoreTest.mdf', MOVE
'LOG_LOGICAL_NAME' TO '/var/opt/mssql/data/Emhip_RestoreTest_log.ldf', CHECKSUM"
$COMPOSE exec -T sqlserver /opt/mssql-tools18/bin/sqlcmd -C -S localhost -U sa -P
"$MSSQL_SA_PASSWORD" -d Emhip_RestoreTest \
-Q "SELECT COUNT(*) AS Guests FROM Guests; SELECT MAX(OccurredAt) AS LastContact FROM
Contacts"
```

Step 4: Clean up and record the result

```
$COMPOSE exec -T sqlserver /opt/mssql-tools18/bin/sqlcmd -C -S localhost -U sa -P
"$MSSQL_SA_PASSWORD" \
-Q "DROP DATABASE [Emhip_RestoreTest]"
rm /opt/emhip-backups/$BACKUP
```

Record the date, the backup file, the guest count and the latest contact date.

8. Logs and troubleshooting

8.1 Where to look

What	Command
Deploy runs	<code>journalctl -u emhip-deploy.service -n 100</code>
Nightly backups	<code>journalctl -u emhip-backup.service -n 50</code>
Timers and next run times	<code>systemctl list-timers 'emhip-*</code>
API	<code>\$COMPOSE logs --tail=200 -f api</code>
Background workers	<code>\$COMPOSE logs --tail=200 -f workers</code>
SQL Server	<code>\$COMPOSE logs --tail=200 sqlserver</code>
Client nginx	<code>\$COMPOSE logs --tail=200 client</code>
Host nginx	<code>/var/log/nginx/access.log</code> and <code>/var/log/nginx/error.log</code>
Container state	<code>\$COMPOSE ps</code>

Container logs belong to the container. A deploy that recreates a container starts a fresh log, so copy anything you need before redeploying. To search recent API errors: `$COMPOSE logs --since 1h api | grep -iE 'fail|exception'`.

8.2 Common problems

Symptom	Likely cause	What to do
Browser shows a certificate or connection error	Host nginx stopped, or certificate expired	<code>systemctl status nginx, nginx -t, certbot certificates</code> (section 10)

Symptom	Likely cause	What to do
502 Bad Gateway on every page	<code>client</code> container down	<code>\$COMPOSE ps</code> , then <code>\$COMPOSE up -d client</code>
App loads but nothing works; <code>/api/health</code> gives 502	<code>api</code> down or restarting, often a failed migration or bad <code>.env</code> value	<code>\$COMPOSE logs --tail=200 api</code> ; fix and redeploy, or restore (section 7). If <code>api</code> is healthy but 502s continue, run <code>\$COMPOSE restart client</code>
Compose says a variable must be set, such as <code>Set JWT_KEY in .env</code>	Variable missing from <code>.env</code>	Add it and run <code>\$COMPOSE up -d</code>
A push did not deploy	Timer stopped, GitHub fetch failing, or backup failed	<code>systemctl list-timers 'emhip-*'</code> , <code>journalctl -u emhip-deploy.service -n 100</code>
Deploy log says "ABORT: sqlserver is running but backup failed"	Disk full, backup folder permissions, or wrong <code>sa</code> password in <code>.env</code>	<code>df -h</code> , <code>ls -ld /opt/emhip-backups</code> , run <code>backup.sh</code> manual to see the error
Deploy log repeats "another deploy is running; skipping"	A long or stuck build	Check <code>systemctl status emhip-deploy.service</code> ; systemd stops it after 45 minutes
New urgent cases do not appear, or no live updates	<code>workers</code> down, or the internal secret rejected	<code>\$COMPOSE ps workers</code> , <code>\$COMPOSE logs workers</code> ; a 401 means <code>api</code> and <code>workers</code> need recreating together (<code>\$COMPOSE up -d api workers</code>)
Urgent list only updates on refresh	WebSocket upgrade not passed by host nginx	Check the <code>/hubs/</code> handling in <code>/etc/nginx/sites-available/emhip</code>
Dashboard counts are zero or old	Materialiser failing, or no <code>Hubs</code> row for the staff hub	Search workers logs for "Report materialization sweep failed"; see section 12.3
Emails do not arrive	Provider set to "Not configured", no from address, template disabled, or provider refusing	Settings, Email tab, Send test email; search logs for "Email not configured" or "Failed to send email"
Links in urgent or overdue emails do not work	<code>PUBLIC_ORIGIN</code> in <code>.env</code> is wrong or missing (the API and workers build email links from it)	Set <code>PUBLIC_ORIGIN=https://emhip.brainshub.co.uk</code> , then <code>\$COMPOSE up -d api workers</code>
Password reset link says invalid or expired	Link older than a day, or sent before the API container was last recreated	Ask for a new link, or reset the password on the Hub Workers screen
Clicking a guest or case does nothing after a deploy	The open tab has old bundles	The app reloads itself once; otherwise refresh the page
Many people get "too many requests" at sign-in	The sign-in rate limit (10 per minute) is shared behind the proxies (known issue)	Wait a minute
One person cannot sign in	Locked for 15 minutes after 5 failures, or account deactivated	Wait, or see section 12.2

Symptom	Likely cause	What to do
Upload fails with 413	File bigger than the host nginx or Settings limit	Check <code>client_max_body_size</code> in the host site config and the maximum file size in Settings
SQL errors about the database being full	SQL Server Express 10 GB database limit	Check the size (section 16) and move to a paid edition via <code>MSSQL_PID</code>
Disk nearly full	Backups, Docker images or logs	<code>df -h</code> , <code>docker system df</code> , <code>du -sh /opt/emhip-backups</code>

9. Restarting services

Task	Command
Restart one service with no config change	<code>\$COMPOSE restart api</code> (or <code>workers</code> , <code>client</code> , <code>sqlserver</code>)
Apply a changed <code>.env</code>	<code>\$COMPOSE up -d</code> (recreates only the services whose settings changed)
Stop the application but keep the database	<code>\$COMPOSE stop api workers client</code>
Start everything	<code>\$COMPOSE up -d</code>
Rebuild from the current checkout	<code>\$COMPOSE build && \$COMPOSE up -d --remove-orphans</code>

What users notice:

- Restarting `api` drops live connections for a few seconds; the app reconnects SignalR and users stay signed in (their token is still valid). Uploads in progress fail.
- Restarting `workers` has no visible effect unless an escalation arrives at that moment; the outbox holds it until the workers are back.
- Restarting `sqlserver` interrupts every request until it is healthy again. Do it at a quiet time.
- After a host reboot, containers start by themselves (`restart: unless-stopped`) and the deploy timer resumes 2 minutes after boot.

Warning: never run `docker compose down -v` on this server. The `-v` deletes the `sqlserver-data` volume, which is the database.

10. TLS and certificates

- The host nginx terminates TLS for `emhip.brainshub.co.uk` on ports 80 and 443 with a Let's Encrypt certificate, renewed automatically by `certbot.timer`. The site configuration is `/etc/nginx/sites-available/emhip`.
- Check the certificate: `certbot certificates` (shows the expiry date).
- Check renewal is scheduled and works: `systemctl list-timers certbot.timer` and `certbot renew --dry-run`.
- After editing the site configuration: `nginx -t && systemctl reload nginx`.
- The app sends HSTS for one year (`client/nginx.conf`), so browsers that have visited will refuse plain HTTP. The site must stay on HTTPS.

- The host nginx configuration is not in the repository. Keep a copy somewhere safe; you need it to rebuild the server (section 7.2).

11. Rotating secrets

The secrets Compose needs are in `/opt/emhip/.env`. Edit it as root (for example `nano /opt/emhip/.env`), keep a copy of every value in the organisation's password manager, and never commit it. Generate new random values with `openssl rand -base64 48`.

Variable	Used by	Effect of changing it
<code>JWT_KEY</code>	API	Every signed-in user is signed out at their next action
<code>INTERNAL_SHARED_SECRET</code>	API and workers	None, if both restart together; if they differ, live urgent-case updates stop
<code>MSSQL_SA_PASSWORD</code>	SQL Server, API, workers, <code>backup.sh</code>	Must be changed inside SQL Server first, or everything loses the database
<code>BOOTSTRAP_ADMIN_EMAIL</code> , <code>BOOTSTRAP_ADMIN_PASSWORD</code>	API	None; used only when there are no users at all
<code>PUBLIC_ORIGIN</code>	API	Changes allowed origins and the links in emails sent by the API
<code>MSSQL_PID</code>	SQL Server	Changes the SQL Server edition (licensing)

11.1 JWT signing key

1. Replace `JWT_KEY` in `.env` with a new value of at least 32 characters.
2. Run `$COMPOSE up -d api`.
3. Sign in to check. Everyone else will be asked to sign in again.

Rotate it whenever someone who had server access leaves, and whenever you need to cut off a deactivated user at once: tokens otherwise stay valid for up to 8 hours.

11.2 Internal shared secret

1. Replace `INTERNAL_SHARED_SECRET` in `.env`.
2. Run `$COMPOSE up -d api workers` so both pick it up together.
3. Raise a test urgent flag on a test guest, or check the workers log for 401 errors after the next escalation.

11.3 SQL Server `sa` password

The `MSSQL_SA_PASSWORD` variable only sets the password when the data volume is first created. Changing `.env` alone breaks the connection strings, so change it in SQL Server first. Choose a password of at least 8 characters with three of upper case, lower case, digits and symbols, and avoid `;` and quote characters because it is placed inside connection strings.

1. Load the current `.env` into the shell (section 1.2).
2. Change the password in SQL Server, replacing `NEW_PASSWORD`:
 - `$COMPOSE exec -T sqlserver /opt/mssql-tools18/bin/sqlcmd -C -S localhost -U sa -P "$MSSQL_SA_PASSWORD" -Q "ALTER LOGIN sa WITH PASSWORD = 'NEW_PASSWORD'"`

3. Put the same value in `MSSQL_SA_PASSWORD` in `.env`.
4. Run `$COMPOSE up -d` to recreate SQL Server (its health check uses the variable), the API and the workers.
5. Run `/opt/emhip/deploy/backup.sh manual` to prove backups still work, then clear the command from your shell history.

11.4 Email and storage credentials

These are not in `.env`. They are stored in the database and edited on the Settings page (Email and Document storage tabs). Rotate them at the provider, enter the new values, save, and use **Send test email** or **Test connection**. The workers read settings through a 5-minute cache, so urgent and overdue emails pick up a change within 5 minutes. When rotating storage keys, the new keys must still reach the same bucket or container, because existing files are read with the saved settings.

11.5 When people leave

- Deactivate their EMHIP account on the Hub Workers screen, then rotate `JWT_KEY` if they must lose access immediately.
- Remove their SSH key from `/root/.ssh/authorized_keys` and their GitHub access (GitHub access means production access).
- If they knew `.env` values, rotate those too.

12. Admin access

Run SQL in this section with an interactive session in the container (load `.env` first, section 1.2), typing `GO` after each statement:

```
$COMPOSE exec sqlserver /opt/mssql-tools18/bin/sqlcmd -C -S localhost -U sa -P "$MSSQL_SA_PASSWORD" -d Emhip
```

Take a manual backup (`/opt/emhip/deploy/backup.sh manual`) before changing data by hand.

12.1 The first admin

When the API starts and the `AspNetUsers` table is empty, it creates one Admin account from `BOOTSTRAP_ADMIN_EMAIL` (default `admin@emhip.local`) and `BOOTSTRAP_ADMIN_PASSWORD`, in the hub `Bootstrap:AdminHubId` (default `22222222-2222-2222-2222-222222222222`, set in `appsettings.json`). If any user exists, nothing is created. The password must meet the sign-in rules (at least 10 characters with an upper-case letter, a lower-case letter and a digit); if it does not, no admin is created and nothing is logged, so correct `.env` and run `$COMPOSE up -d api`. After the first sign-in:

1. Open **Hub Workers**, find the admin account and use **Reset password** to set a new password. There is no separate change-password screen.
2. Create a named Admin account for each administrator. The Hub ID field defaults to your own hub.
3. Once a named admin can sign in, deactivate the bootstrap account (accounts cannot be deleted).
4. Check that a `Hubs` row exists for the hub (section 12.3).

12.2 Recovering access

Try these in order. The admin password reset on the Hub Workers screen does not clear a logout by itself.

Option 1: another admin resets the password

Any account with `admin.manageusers` can set a new password on the Hub Workers screen.

Option 2: forgot password

Use **Forgot password** on the sign-in page, if email is set up. The link is valid for one day and stops working if the API container is recreated before it is used.

Option 3: account locked

After 5 failed attempts the account is locked for 15 minutes. Wait, or clear the lockout:

```
UPDATE AspNetUsers SET LockoutEnd = NULL, AccessFailedCount = 0 WHERE NormalizedEmail =  
UPPER('admin@example.org');
```

Option 4: account deactivated

```
UPDATE AspNetUsers SET IsActive = 1 WHERE NormalizedEmail = UPPER('admin@example.org');
```

Option 5: no one with the Admin role can sign in

If someone else can still sign in, give that person the Admin role, then ask them to sign out and in again:

```
INSERT INTO AspNetUserRoles (UserId, RoleId)  
SELECT u.Id, r.Id FROM AspNetUsers u CROSS JOIN AspNetRoles r  
WHERE u.NormalizedEmail = UPPER('someone@example.org') AND r.NormalizedName = 'ADMIN';
```

Option 6: nobody can sign in and email is not set up

Set a new password hash directly. On a machine with the .NET SDK, create a small program that prints an ASP.NET Core Identity hash:

```
dotnet new console -o hashpw && cd hashpw  
dotnet add package Microsoft.Extensions.Identity.Core
```

Replace `Program.cs` with:

```
using Microsoft.AspNetCore.Identity;  
Console.WriteLine(new PasswordHasher<object>().HashPassword(new object(), args[0]));
```

Run it with the new password as the argument (`dotnet run -- 'the-new-password'`), paste the output into the statement below, and run it on the server. Then sign in and set a fresh password on the Hub Workers screen.

```
UPDATE AspNetUsers  
SET PasswordHash = 'PASTE_HASH_HERE', SecurityStamp = CONVERT(nvarchar(36), NEWID()),  
    LockoutEnd = NULL, AccessFailedCount = 0, IsActive = 1  
WHERE NormalizedEmail = UPPER('admin@example.org');
```

12.3 Making sure the hub exists

Every staff account and guest carries a hub id, but there is no screen for hubs. The dashboard counts are only calculated for hubs that have a row in the `Hubs` table. Check that every hub id in use has one:

```
SELECT DISTINCT u.HubId, h.Code, h.Name FROM AspNetUsers u LEFT JOIN Hubs h ON h.Id =  
u.HubId;
```

If `Code` is empty for a hub id in use, add the row (use the hub id from the query; codes must be unique and at most 20 characters):

```
INSERT INTO Hubs (Id, Name, Code) VALUES ('22222222-2222-2222-2222-222222222222', 'Main Hub',  
'HUB001');
```

Counts appear within 5 minutes.

13. Setting up email

Email is configured in the app, not on the server. Until it is, messages are logged and not sent, so password reset and the account-created, urgent-case and overdue emails do nothing.

1. Sign in as an Admin (needs `settings.manage`) and open **Settings**, then the **Email** tab.
2. Choose the provider and fill in its fields:
 - **SMTP server**: host, port (587 for STARTTLS, 465 for SSL on connect, 25 for an internal relay), encryption, username and password.
 - **Amazon SES**: region (default `eu-west-2`), access key and secret key. The from address must be a verified identity in that region.
 - **Mailgun**: domain, API key and region. Use the EU region for an EU-hosted domain.
3. Set the from address (required), from name and optional reply-to address, and save.
4. Use **Send test email** to send a real message to yourself.
5. Review the wording on the **Email templates** tab: password reset, account created, urgent case raised, contact overdue and test email. Each can be switched off.
6. Choose whether to email workers when a case becomes urgent and about overdue contacts (both on by default).

Also:

- Set up SPF, DKIM and DMARC for the sending domain with your DNS provider, or messages may be marked as spam.
- Make sure `PUBLIC_ORIGIN` in `.env` is `https://emhip.brainshub.co.uk`; it is the base of the password-reset link.
- Provider credentials are stored in the database in plain text and never shown again on screen. Put a data processing agreement in place with the provider (document 09).

14. Setting up document storage

By default, documents are stored on the server in `/opt/emhip-documents` (the API sees it as `/var/emhip/documents`) and included in every backup. To use cloud storage instead:

1. Create the bucket or container at the provider, in a UK or EU region, with credentials limited to it.
2. In **Settings**, open **Document storage** and choose the provider:
 - **Amazon S3**: bucket, region, access key, secret key.

- **S3-compatible** (Contabo, MinIO and similar): the same, plus the service URL (for example `https://eu2.contabostorage.com`); keep path-style URLs on.
- **Azure Blob Storage**: connection string and container (default `emhip-documents`).
- **Google Cloud Storage**: bucket and the full service-account JSON.

3. Use **Test connection**, which writes and deletes a small probe file, then save.

4. Upload a test document to a test guest and download it again.

Only new uploads go to the new provider. Existing files stay where they were and are read with the saved settings for that provider, so keep `/opt/emhip-documents`, its backups and the old provider's credentials. `backup.sh` does not copy cloud-stored files; turn on versioning or backups at the provider.

Upload limits: the **Uploads** tab sets the maximum file size (default 25 MB) and the allowed file types. The container nginx allows up to 512 MB and the API up to 500 MB; the host nginx site must also allow the size you choose.

15. Importing legacy data

The data-migration import loads guests from a CSV export of the previous system (InForm). It is on the **Data migration** tab of **Settings** and needs the `admin.manageusers` permission (Admin by default). The API endpoints are `GET /api/admin/migration/guest-template` and `POST /api/admin/migration/guests?dryRun=true` (a dry run unless `dryRun=false`); files can be up to 100 MB.

What it imports: guests with contact details, status and pathway, demographics, one note per row and one DIALOG assessment per row, keeping the original dates. It does not import contacts, scheduled contacts or risk history.

Column rules	Detail
Required	<code>first_name</code> , <code>last_name</code> , <code>date_of_birth</code>
Dates	<code>yyyy-MM-dd</code> , <code>dd/MM/yyyy</code> , <code>d/M/yyyy</code> , <code>dd-MM-yyyy</code> or <code>MM/dd/yyyy</code> , tried in that order, so <code>03/04/1990</code> is 3 April
<code>status</code>	<code>New</code> , <code>Active</code> or <code>OnHold</code> ; anything else becomes <code>New</code> with a warning
<code>pathway</code>	<code>MentalWellbeing</code> , <code>ClinicalSupport</code> or <code>CommunityRecovery</code> (spaces ignored)
<code>dialog_scores</code>	11 numbers from 1 to 7, separated by spaces or semicolons
<code>legacy_id</code>	Makes the import repeatable: a row whose <code>legacy_id</code> already exists updates that guest
Other columns	See the template; unknown columns are reported and ignored

Procedure:

1. Take a backup: `/opt/emhip/deploy/backup.sh pre-import`.
2. Sign in as an account that has `admin.manageusers` and belongs to the hub the guests should go to. Guests are imported into the signed-in user's hub.
3. Download the template and map the export to its columns.
4. Run a dry run and fix every reported problem. The dry run counts every valid row as "created", even rows that would update an existing guest.
5. Run the real import. Rows that still have errors are skipped and the rest are saved together.
6. Spot-check a few imported guests, and check the dashboard after 5 minutes.

7. Delete the CSV from wherever it was stored; it holds special-category data.

16. Monitoring suggestions

Nothing monitors EMHIP at present. In order of value:

- **Uptime:** an external check of `https://emhip.brainshub.co.uk/api/health` every few minutes, alerting on anything but 200.
- **Certificate expiry:** alert when fewer than 14 days remain.
- **Backups:** alert if the newest `emhip-*.bak.gz` in `/opt/emhip-backups` is more than 26 hours old, or if `emhip-backup.service` fails (`systemctl --failed`).
- **Disk:** alert at 80% on the root filesystem; backups, Docker images and the database all live there.
- **Deploys:** alert when `emhip-deploy.service` fails (it then appears in `systemctl --failed`) or its log contains `ABORT`.
- **Containers:** alert if any of the four services is not running or keeps restarting.
- **Database size:** the Express edition stops at 10 GB per database. `AuditEvents` grows with every guest page view.
- **Background work:** unprocessed outbox rows and stale dashboard snapshots show the workers are stuck.

Useful SQL for the last two:

```
-- Data file size in MB (Express limit is 10240)
SELECT SUM(CAST(size AS bigint)) * 8 / 1024 AS DataMb FROM sys.database_files WHERE type_desc = 'ROWS';

-- Largest tables by row count
SELECT TOP 10 t.name, SUM(p.rows) AS TotalRows
FROM sys.tables t JOIN sys.partitions p ON p.object_id = t.object_id AND p.index_id IN (0, 1)
GROUP BY t.name ORDER BY TotalRows DESC;

-- Outbox backlog: should be close to 0
SELECT COUNT(*) AS Pending, SUM(CASE WHEN Error IS NOT NULL THEN 1 ELSE 0 END) AS WithErrors
FROM OutboxMessages WHERE ProcessedAt IS NULL;

-- Dashboard snapshots: RefreshedAt should be under 10 minutes old
SELECT HubId, RefreshedAt FROM DashboardSnapshots_ReadModel;
```

17. Routine checks

When	Check	How
Daily	Site is up	Open the site, or <code>curl /api/health</code> (section 4)
Daily	Last night's backup exists	<code>ls -lt /opt/emhip-backups</code> and look for today's scheduled file
Daily	No failed deploy or backup	<code>systemctl --failed</code> and the deploy journal
Weekly	Disk space	<code>df -h</code> and <code>du -sh /opt/emhip-backups</code>
Weekly	All containers healthy, none restarting	<code>\$COMPOSE ps</code>

When	Check	How
Weekly	Errors in API and worker logs	<code>\$COMPOSE logs --since 168h api</code> and <code>workers</code> , searching for exceptions
Weekly	Workers keeping up	Outbox and snapshot queries (section 16)
Monthly	Test restore	Section 7.3; record the result
Monthly	Certificate and renewal	<code>certbot certificates</code> and <code>certbot renew --dry-run</code>
Monthly	Database size against the 10 GB limit	SQL in section 16
Monthly	Operating system updates	<code>apt list --upgradable</code> ; reboot if <code>/var/run/reboot-required</code> exists
Monthly	Base images	<code>\$COMPOSE build --pull</code> and <code>\$COMPOSE pull sqlserver</code> at a quiet time, then <code>\$COMPOSE up -d</code>
Monthly	Off-site backup copy	Confirm the latest copy arrived (once set up, section 6.2)
Quarterly	Staff accounts	Deactivate leavers on the Hub Workers screen
Quarterly	Roles and permissions	Review the Roles & Permissions screen
Quarterly	Server and GitHub access	Review <code>authorized_keys</code> and repository collaborators; rotate secrets after changes (section 11)
Yearly	Retention review	Data Quality report, "past retention" count (document 04)